

1. Motivation
2. ADT SortedMap
3. Implémentations simples avec un tableau
4. Applications :
 - a. base de vols d'avion
 - b. trouver les points maximaux
5. Peut-on faire mieux? “Skiplist”

Motivation pour l'ADT SortedMap

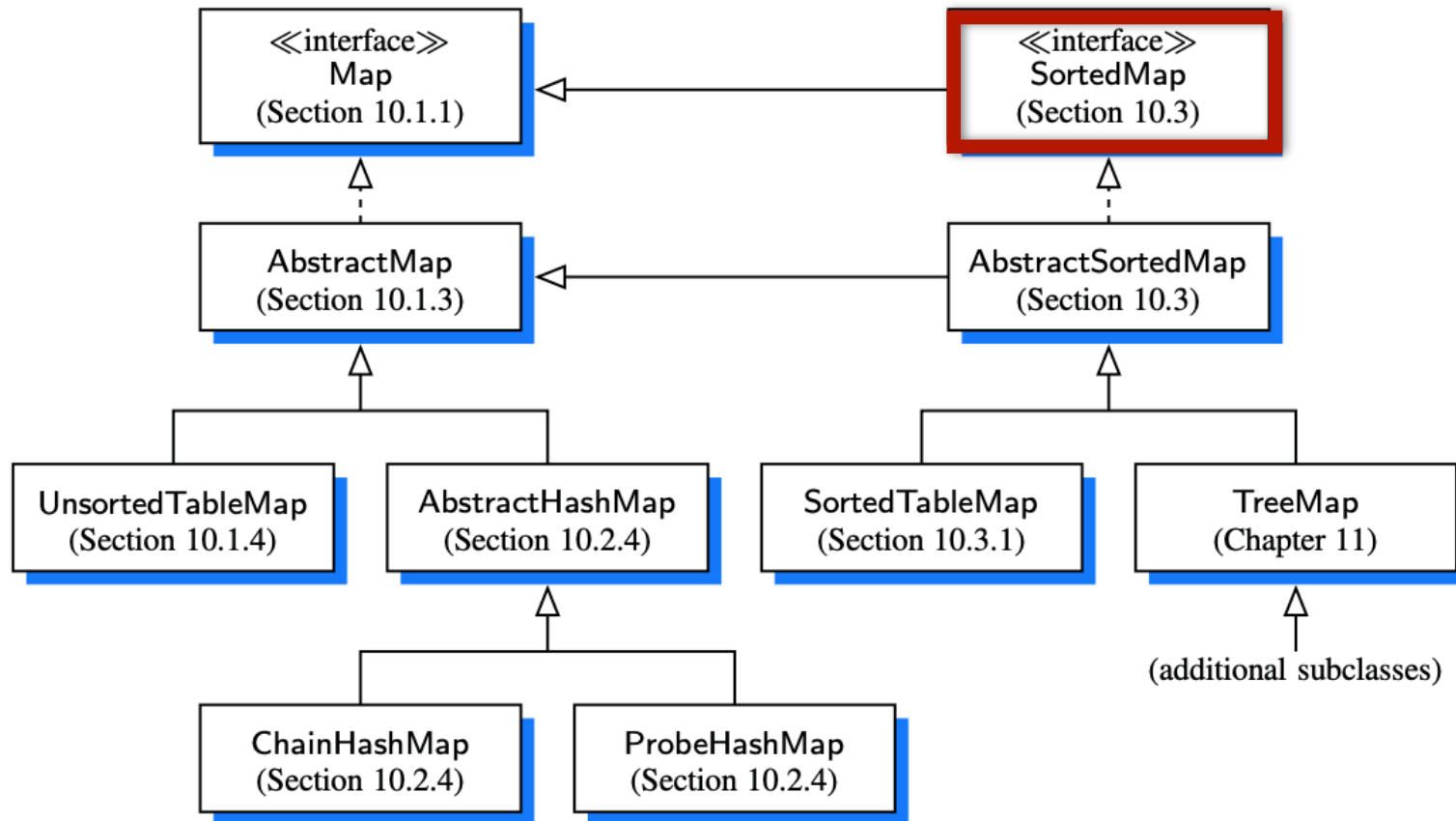
Pour motiver l'utilisation d'une **Map** triée, pensez à des systèmes qui conservent les informations sur des événements qui se produisent dans le temps, comme des transactions financières horodatées marquant l'occurrence de chaque événement. En fait, n'importe quel élément qu'on voudrait chercher en fonction d'un ou plusieurs critères, comme la date et l'heure de vols d'avion ou encore le prix, etc.

On peut définir des clés avec plusieurs critères :

$$k = [\text{origine, destination, date, heure}]$$

Une **Map** (non triée) ne fournit aucun moyen d'obtenir une liste de tous les événements classés selon le temps auquel ils se produisent, ou, dans le cas de vols, de chercher ceux qui sont disponibles dans un intervalle de dates ou d'heures, à moins de parcourir toutes les entrées 😞

Hiérarchie des types de Map



Dans ce module, nous utilisons une structure arborescente de recherche pour implémenter efficacement l'ADT `SortedMap`.

L'ADT `SortedMap` inclut des fonctionnalités garantissant l'itération sur les clés dans l'ordre et prenant en charge des recherches supplémentaires, telles que `higherEntry( k )` et `subMap( k1, k2 )`.

Les arbres binaires sont une excellente structure de données pour stocker les entrées d'une `SortedMap`, en supposant que nous ayons une relation d'ordre définie sur les clés.

Rappel, les 3 opérations de gestion de l'ADT `Map` :

<code>get(k)</code>	Retourne la valeur v associée à la clé k , si elle existe, sinon retourne <code>null</code>
<code>put(k, v)</code>	S'il n'y a pas d'entrée avec la clé k , alors ajoute l'entrée $[k, v]$ et retourne <code>null</code> . Sinon, remplace avec v la valeur existante de l'entrée avec la clé k et retourne l'ancienne valeur
<code>remove(k)</code>	Retire l'entrée avec la clé k et retourne sa valeur. S'il n'y a pas d'entrée avec la clé k , retourne <code>null</code>

Voir le code de :

- `SortedMap.java` et
- `AbstractSortedMap.java`.

L'ADT SortedMap

L'ADT `Map` permet à un utilisateur de rechercher la valeur associée à une clé donnée, mais la recherche de cette clé correspond à ce qu'on appelle une recherche exacte. Ici, nous allons introduire une extension pour une `Map` triée qui inclut tous les comportements de la `Map` standard, plus les suivants :

<code>firstEntry()</code>	Retourne l'entrée avec la plus petite clé (ou <code>null</code> si la <code>Map</code> est vide)
<code>lastEntry()</code>	Retourne l'entrée avec la plus grande clé (ou <code>null</code> si la <code>Map</code> est vide)
<code>ceilingEntry(k)</code>	Retourne l'entrée avec la plus petite clé qui est plus grande ou égale à <code>k</code> (ou <code>null</code> si une telle entrée n'existe pas)
<code>floorEntry(k)</code>	Retourne l'entrée avec la plus grande clé qui est plus petite ou égale à <code>k</code> (ou <code>null</code> si une telle entrée n'existe pas)
<code>lowerEntry(k)</code>	Retourne l'entrée avec la plus grande clé qui est strictement plus petite que <code>k</code> (ou <code>null</code> si une telle entrée n'existe pas)
<code>higherEntry(k)</code>	Retourne l'entrée avec la plus petite clé qui est strictement plus grande que <code>k</code> (ou <code>null</code> si une telle entrée n'existe pas)
<code>subMap(k₁, k₂)</code>	Retourne une collection itérable sur les entrée avec une clé plus grande ou égale à <code>k₁</code> , mais strictement plus petite que <code>k₂</code> .

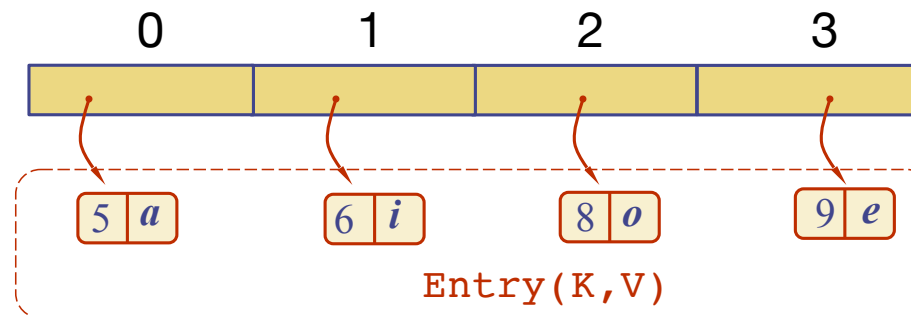
👉 Notez que ces méthodes sont incluses dans l'interface `java.util.NavigableMap` (qui étend l'interface plus simple `java.util.SortedMap`).

Implémentation de *SortedMap* avec une *ArrayList* triée

Nous pouvons implémenter Map avec une liste triée

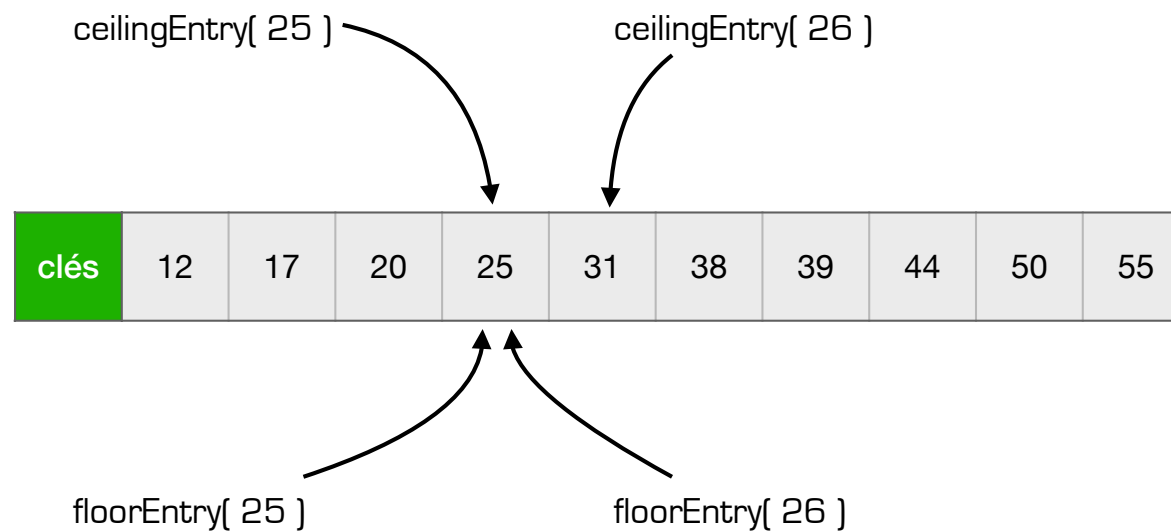
Nous stockons les éléments dans l'ordre des clés

ArrayList index:



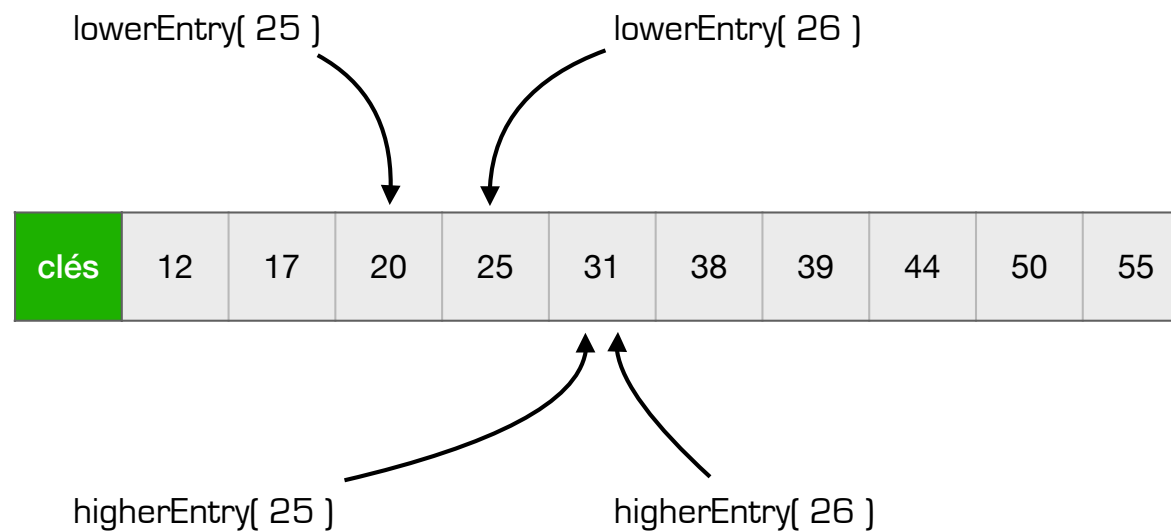
```
{('B', 4)('K', 9)('U', 2)('V', 8)} size = 4
findIndex testing
findIndex( 'B', 0, 3 ) = 0
findIndex( 'K', 0, 3 ) = 1
findIndex( 'U', 0, 3 ) = 2
findIndex( 'V', 0, 3 ) = 3
findIndex( 'A', 0, 3 ) = 0 ← index le plus à gauche avec une clé >= 'A'
findIndex( 'Y', 0, 3 ) = 4 ← index le plus à gauche avec une clé >= 'Y'
findIndex( 'L', 0, 3 ) = 2 ← index le plus à gauche avec une clé >= 'L'
```

Visualisation des fonctions "Entry"



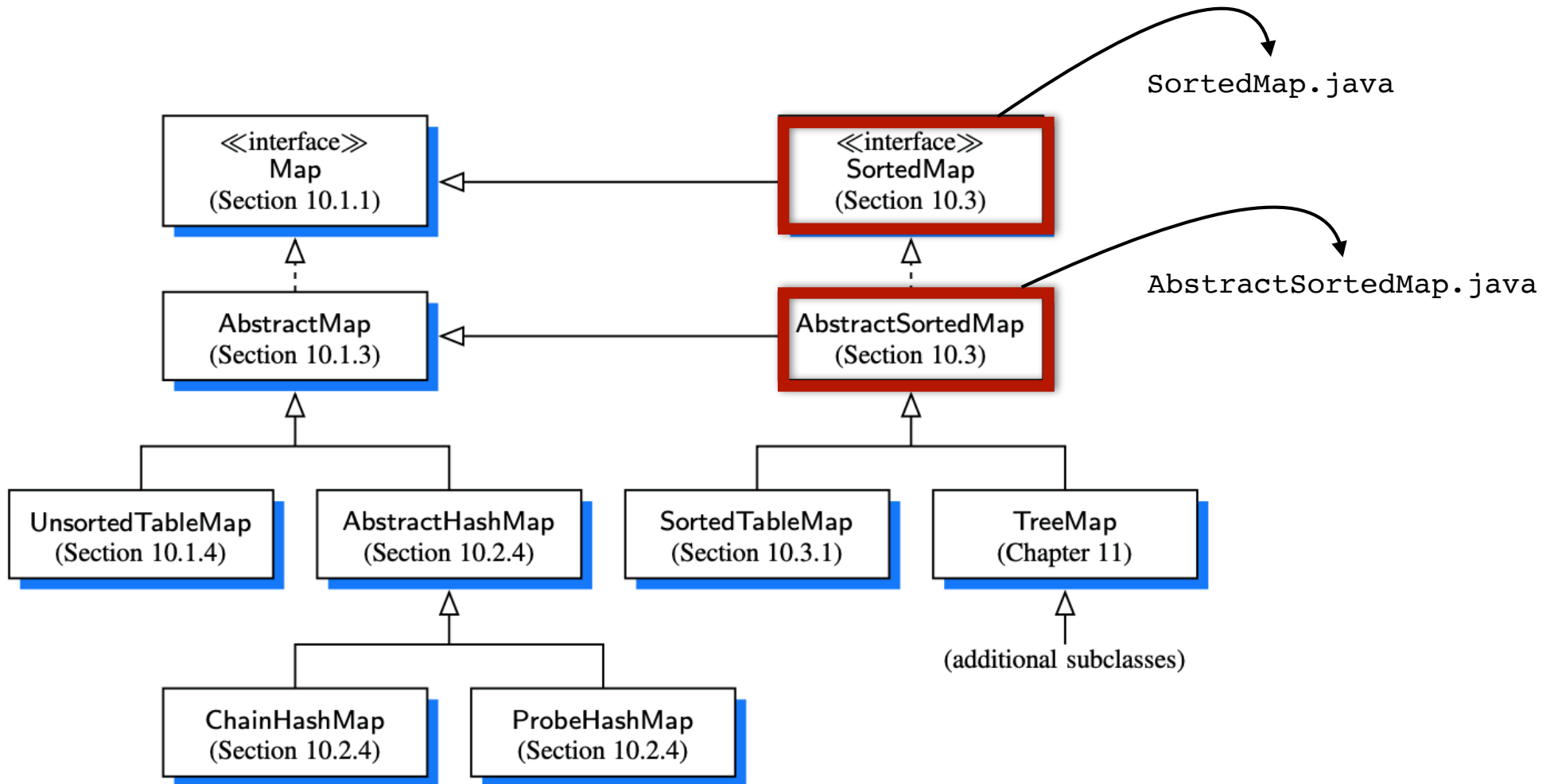
`ceilingEntry( k )`
`floorEntry( k )`

Visualisation des fonctions "Entry"



`lowerEntry( k )`
`higherEntry( k )`

Hiérarchie des types de Map



Performances

Structure de données	put (millisec.)	get (millisec.)	remove (millisec.)
UnsortedTableMap	pire cas $O(n)$	pire cas $O(n)$	pire cas $O(n)$
200K clés	28 400	25 700	46 100
SortedTableMap	pire cas $O(n)$	pire cas $O(\log n)$	pire cas $O(n)$
200K clés	528	68	467
ProbeHashMap	esp. $O(1)$	esp. $O(1)$	esp. $O(1)$
200K clés	51	19	20

👉 Pourquoi put et remove sont si lents avec SortedTableMap ?

👉 La différence entre UnsortedTableMap et SortedTableMap provient de la recherche dichotomique en $O(\log n)$ pour cette dernière.

*Temps d'exécution sur M1 Ultra

Performances de SortedTableMap

<code>size</code>	$O(1)$
<code>get</code>	$O(\log n)$ recherche dichotomique
<code>put</code>	$O(n)$ insertion triée dans <code>ArrayList</code> $O(\log n)$ si la Map possède déjà un élément avec la clé
<code>remove</code>	$O(n)$ suppression dans <code>ArrayList</code>
<code>firstEntry,</code> <code>lastEntry</code>	$O(1)$ accès au premier et dernier élément d'un <code>ArrayList</code> , respectivement
<code>ceilingEntry,</code> <code>floorEntry,</code> <code>lowerEntry,</code> <code>higherEntry</code>	$O(\log n)$ recherche dichotomique
<code>subMap</code>	$O(s + \log n)$ où s entrées sont retournées et $\log n$ pour la recherche dichotomique du début de l'intervalle
<code>entrySet,</code> <code>keySet,</code> <code>values</code>	$O(n)$

Application : base de vols d'avion

Les vols utilisent des clés :

$$k = [\text{origine}, \text{destination}, \text{date}, \text{heure}]$$

Les valeurs sont les détails des vols : numéro, sièges, temps, prix.

Les utilisateurs d'un site web de vols permet une flexibilité dans les recherches. Exemple] une flexibilité de dates.

Solution : SortedMap

Opérations :

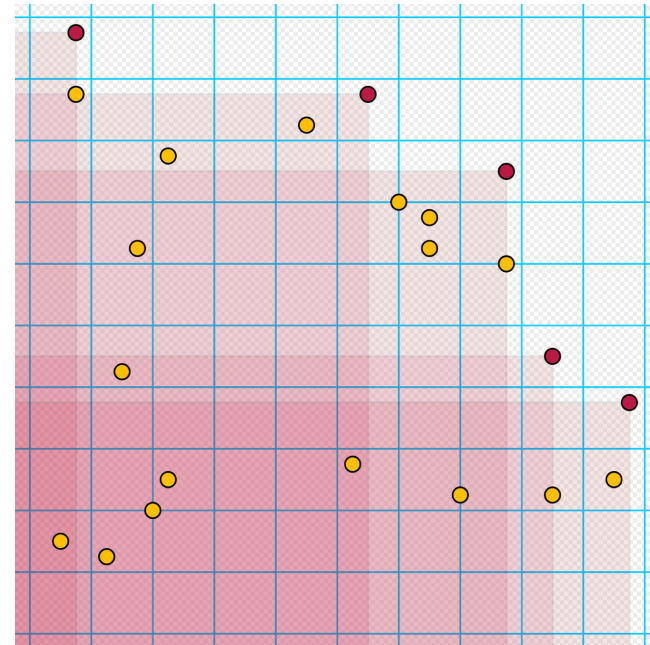
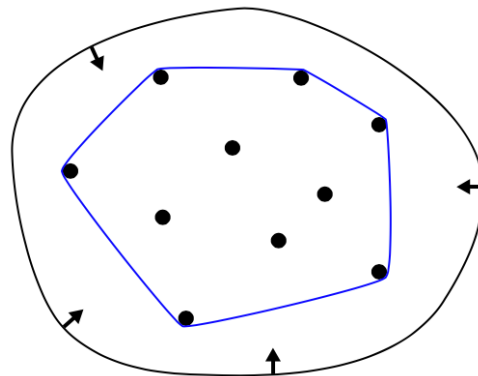
- `ceilingEntry( k )` => premier vol à ou après une date
- `subMap( k1, k2 )` => tous les vols entre 2 dates

Les requêtes sont traitées sur des intervalles sur l'ordre des clés et non pas des "lookup" exacts.

Application : trouver l'ensemble des points maximaux

En géométrie computationnelle, un point p dans un ensemble fini de points S est dit maximum ou non dominé s'il n'y a pas d'autre point q dans S dont les coordonnées sont toutes supérieures ou égales aux coordonnées correspondantes de p .

Trouver l'ensemble des points maximaux d'un ensemble de points S consiste à identifier tous les points maximaux de S . Ce problème (appelé maxima-set en anglais) est une variante des problèmes d'enveloppe convexe (convex hull) et enveloppe convexe orthogonale. Cela équivaut à trouver la frontière de Pareto d'un ensemble de points.



Les cinq points rouges sont les points maximaux de l'ensemble de tous les points. Les régions ombrées indiquent les points dominés par chacun des cinq points maximaux.

David Eppstein sur Wikipedia

L'enveloppe convexe ou la fermeture convexe d'une forme est le plus petit ensemble convexe qui la contient.

“Maximaset” de 2 variables : qualité-prix (imaginez chercher un voyage céleste excitant au meilleur prix)

La vie est pleine de compromis ! On doit souvent négocier les mesures qualité-prix. Le fameux "good, fast, cheap, pick two ; pour réaliser une tâche de programmation !

Supposez une agence de voyage offrant des voyages célestes. On voudrait maximiser la sensation et minimiser le coût.

On peut modéliser un tel problème (de compromis) en utilisant des paires (clé, valeur) pour tenir compte des deux paramètres ; dans ce cas les paires (prix, sensation).

Notez que certains voyages sont strictement “plus tripatifs” que d'autres selon ces mesures. Par exemple, un voyage vers Mars (30 000\$, 100) est strictement “plus trippant” qu'un tour de Lune (40 000\$, 50). Par contre, certains voyages ne sont pas strictement “dominés” par un autre. Par exemple, le voyage (30 000\$, 100) peut être “mieux” ou “pire” qu'un voyage (40 000\$, 110), selon qu'on puisse ou non dépenser cet argent.

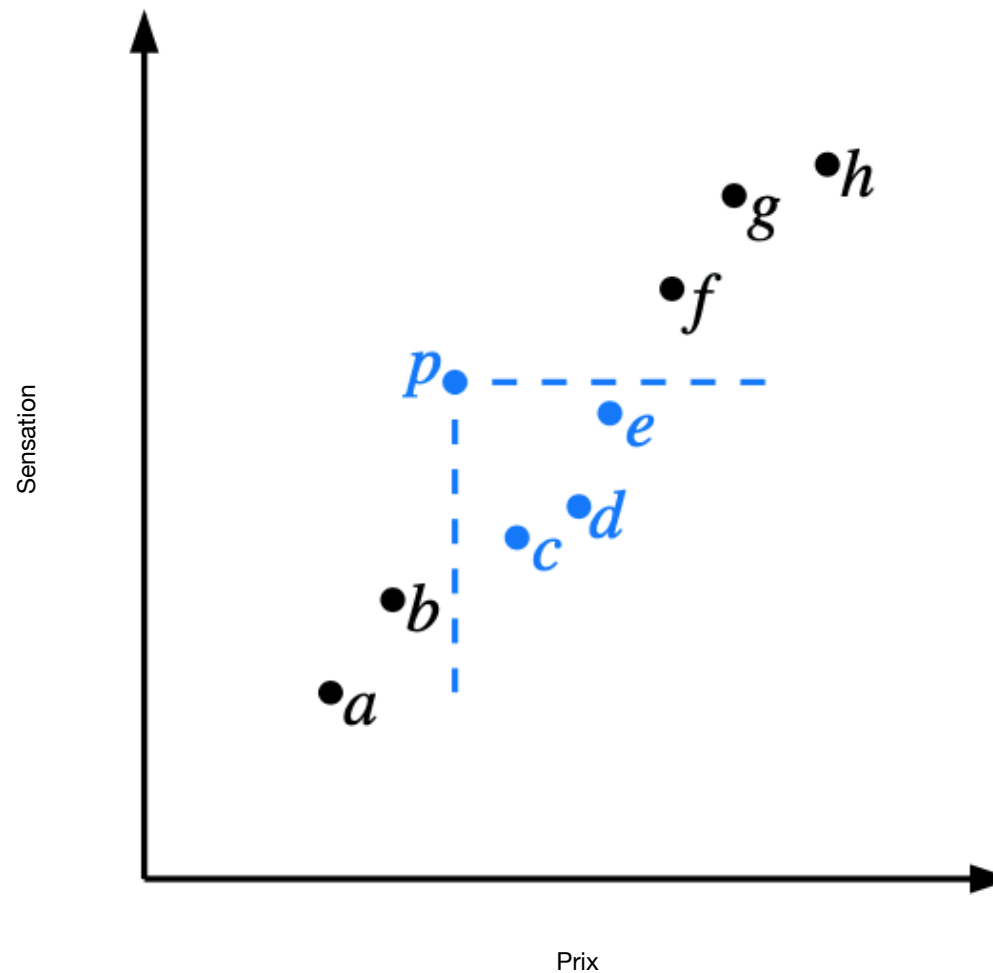
Formellement, on dit qu'un couple $[a, b]$ **domine** le couple $[c, d] \neq [a, b]$ si $a \leq c$ et $b \geq d$, ou autrement dit si le coût du premier couple n'est pas supérieur et est d'aussi bonne qualité.

Une paire $[a, b]$ est appelée **paire maximale** si elle n'est dominée par aucune autre paire.

On s'intéresse au maintien de l'ensemble des paires maximales (**points maximaux**) d'une collection de paires/points. On ajoute de nouvelles paires et interroge cette collection pour un coût donné, d ; ceci nous permet de trouver le voyage le plus sensationnel qui ne coûte pas plus que d \$.



“Maximaset” de 2 variables

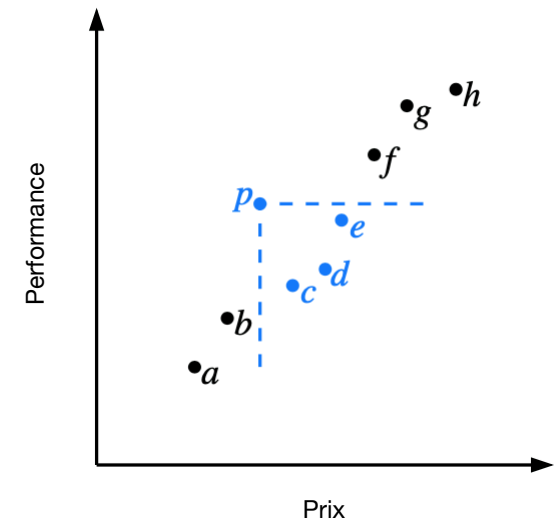


Le compromis (coût, sensation) avec des paires représentées par des points dans le plan.

Notez que le point *p* est strictement “meilleur” que les points *c*, *d* et *e*, mais peut être “meilleur” ou “pire” que les points *a*, *b*, *f*, *g* et *h*, selon le prix à payer. Ainsi, si nous devons ajouter *p* à notre ensemble, nous pourrions supprimer les points *c*, *d* et *e*, mais pas les autres.

“Maximaset” de 2 variables

On peut stocker l'ensemble des paires maximales dans une `SortedMap` avec le coût dans le champ clé et la sensation dans le champ valeur. On peut implémenter les opérations `add( c, p )`, qui ajoute une nouvelle entrée, et `best( c )`, qui renvoie l'entrée de meilleure sensation de toutes celles dont le coût est au plus `c`.



Voir le code de `PriceQuality.java`

Voir le code de `Trips.java`

```

a [java] {<2500:90>}
b [java] {<2500:90>, <3000:100>}
c [java] {<2500:90>, <3000:100>, <3500:120>}
d [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>}
e [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>}
f [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>}
g [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>, <3700:155>}
h [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>, <3700:155>, <3900:160>}
élimine c, d et e : p [java] {<2500:90>, <3000:100>, <3200:145>, <3600:150>, <3700:155>, <3900:160>}

[java] Proposed trip: <2001:300>
[java] Time to filter 1 dominant trip over 500 millions data points is 18106 milliseconds.
[java] [<2001:300>]

```

Est-ce qu'on peut faire mieux ?

Une implémentation avec une liste triée est pas mal sympa !

Malheureusement, si on implémente la `SortedMap` à l'aide de la classe `SortedTableMap`, l'ajout d'une paire en pire des cas est dans $O(n)$.

Ceci vient du fait que malgré le bénéfice de la recherche dichotomique en $O(\log n)$, les méthodes de mise à jour (**put** et **remove**) restent en $O(n)$.

Pourquoi ?

Comment pourrait-on obtenir des méthodes de mises à jour plus efficaces ?

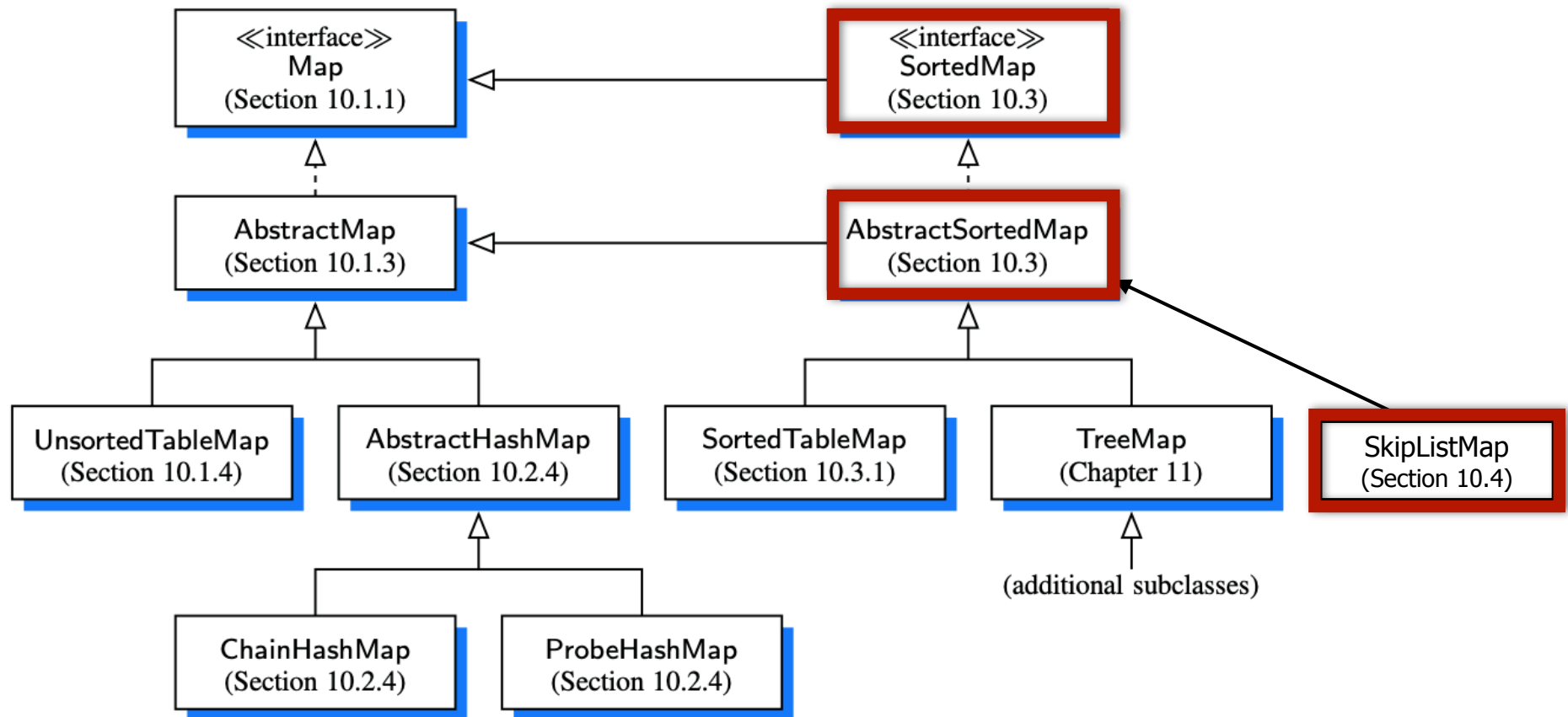
Serait-il possible de combiner les insertions et suppressions rapides d'une liste chaînée avec la recherche dichotomique d'une clé ?

Mais comment faire ça ?

Imaginez une implémentation de `Map` avec des méthodes de mise à jour (**put** et **remove**) en $O(\log n)$ et **add**[`c, p`] en $O((1 + r) \log n)$, où r est le nombre de points à supprimer.

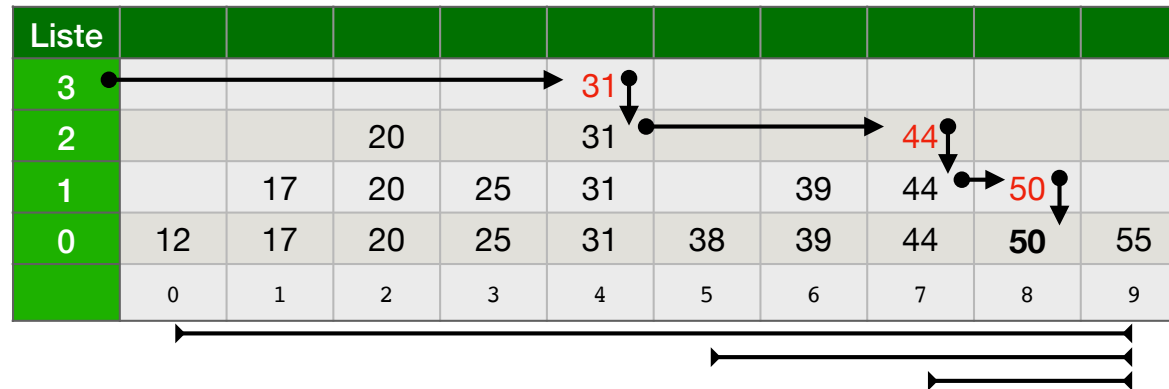
Et bien, il existe une telle structure de données ! C'est la **SkipList**

Hiérarchie des types de Map



Arbre parfaitement divisé selon une liste d'entrées

[à chaque niveau on a les valeurs moyennes des intervalles d'une recherche dichotomique]



`binarySearch( 50 ); // in  $O(\log n)$`

Pour chercher la clé **50**, la recherche dichotomique suggère :

- intervalle 0 à 9 (valeur cherchée **50** > valeur milieu **31**)
- intervalle 5 à 9 (valeur cherchée **50** > valeur milieu **44**)
- intervalle 7 à 9 (valeur cherchée **50** = valeur milieu **50**)

Bingo ! valeur recherchée est à l'indice 8 !

`skipSearch( 50 ); // in  $O(\log n)$`

Pour chercher la clé **50** dans la liste de niveau 0 :

- Traverse liste 3, valeur cherchée **50** > **31**
- Traverse liste 2 à partir de **31**, valeur cherchée **50** > **44**
- Traverse liste 1 à partir de **44**, valeur cherchée **50** = **50**

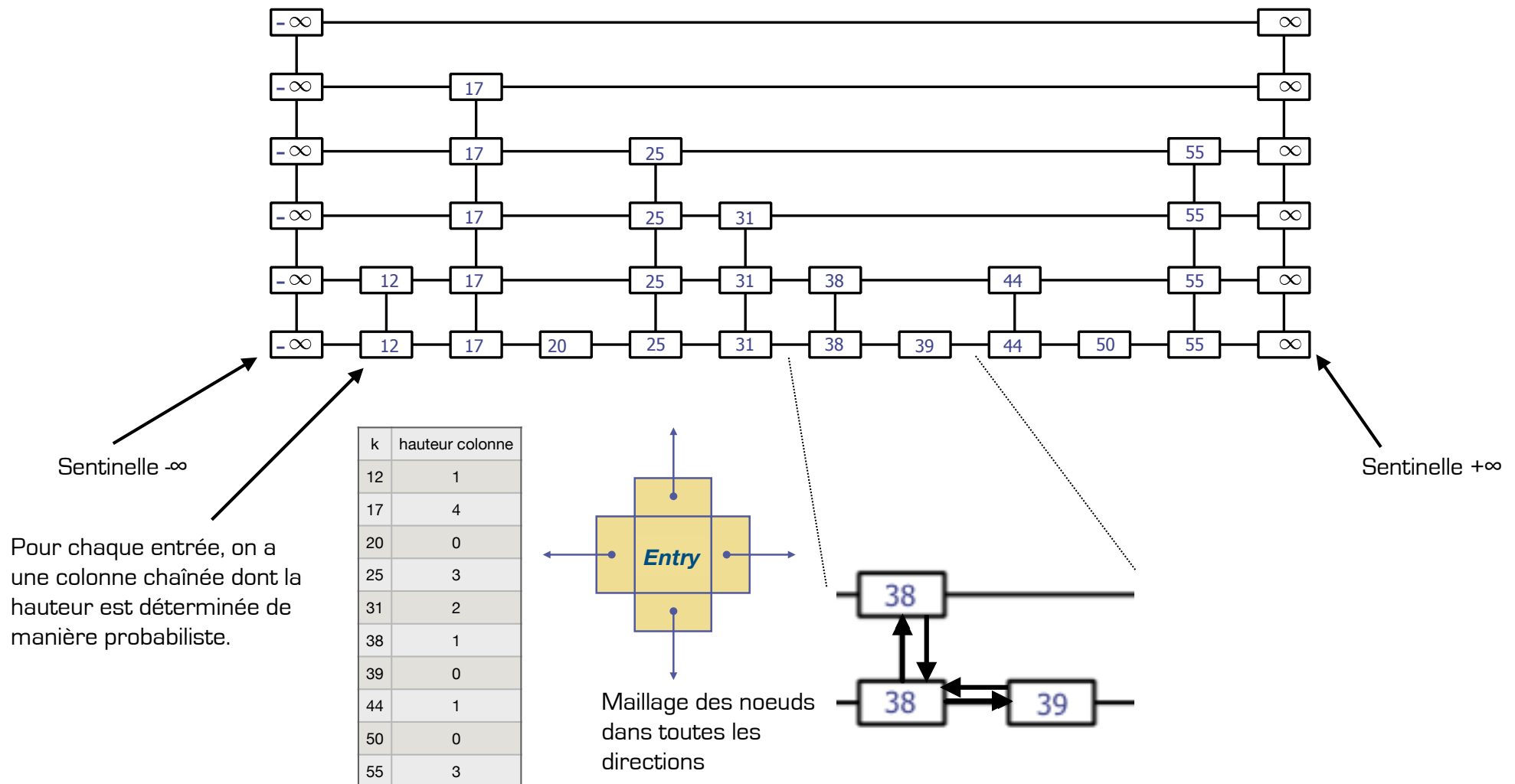
Bingo ! valeur cherchée est à l'indice 8 !

👉 On a ici des listes qui représentent une décomposition binaire parfaite.

Décomposer une liste de cette manière n'est pas trop difficile mais peut-on la garder ainsi après une série d'opérations de mise à jour ?

Exemple d'une **skipList**

- Remarquez que les listes sont aussi reliées par des colonnes, une pour chaque entrée
- Les sentinelles des listes créent deux colonnes sentinelles à gauche ($-\infty$) et à droite ($+\infty$)



Voir le code de `SkipListMap.java`

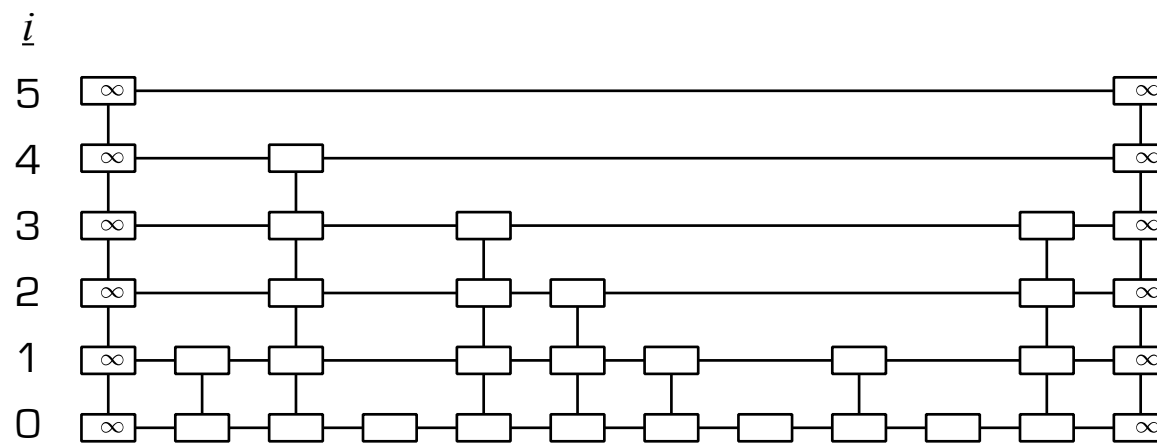
Hauteur d'une **SkipList**

Intuitivement, les listes sont établies de telle sorte que la liste au niveau $i+1$ contienne un certain nombre d'entrées du niveau i . Cependant, ce nombre qui devrait idéalement correspondre à la moitié du nombre d'entrées d'une liste à la suivante n'est pas imposé en tant que propriété explicite; on utilise plutôt les probabilités pour déterminer le nombre de valeurs dans chaque liste.

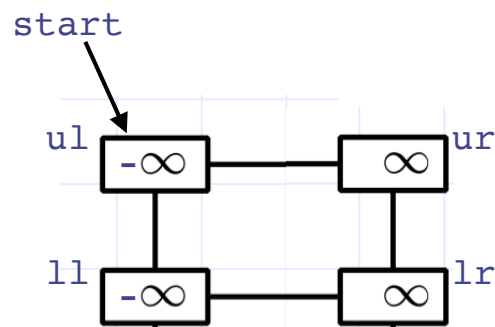
La méthode d'insertion choisie les entrées au niveau $i+1$ au hasard parmi les entrées au niveau i avec une probabilité de $1/2$.

Ainsi, on s'attend à ce que le niveau 1 contienne environ $n/2$ entrées, le niveau 2 environ $n/4$ entrées et, en général, le niveau i environ $n/2^i$ entrées

En conséquence, la hauteur espérée, h , d'une **SkipList** est $\log n$.



La **SkipList** initialement vide



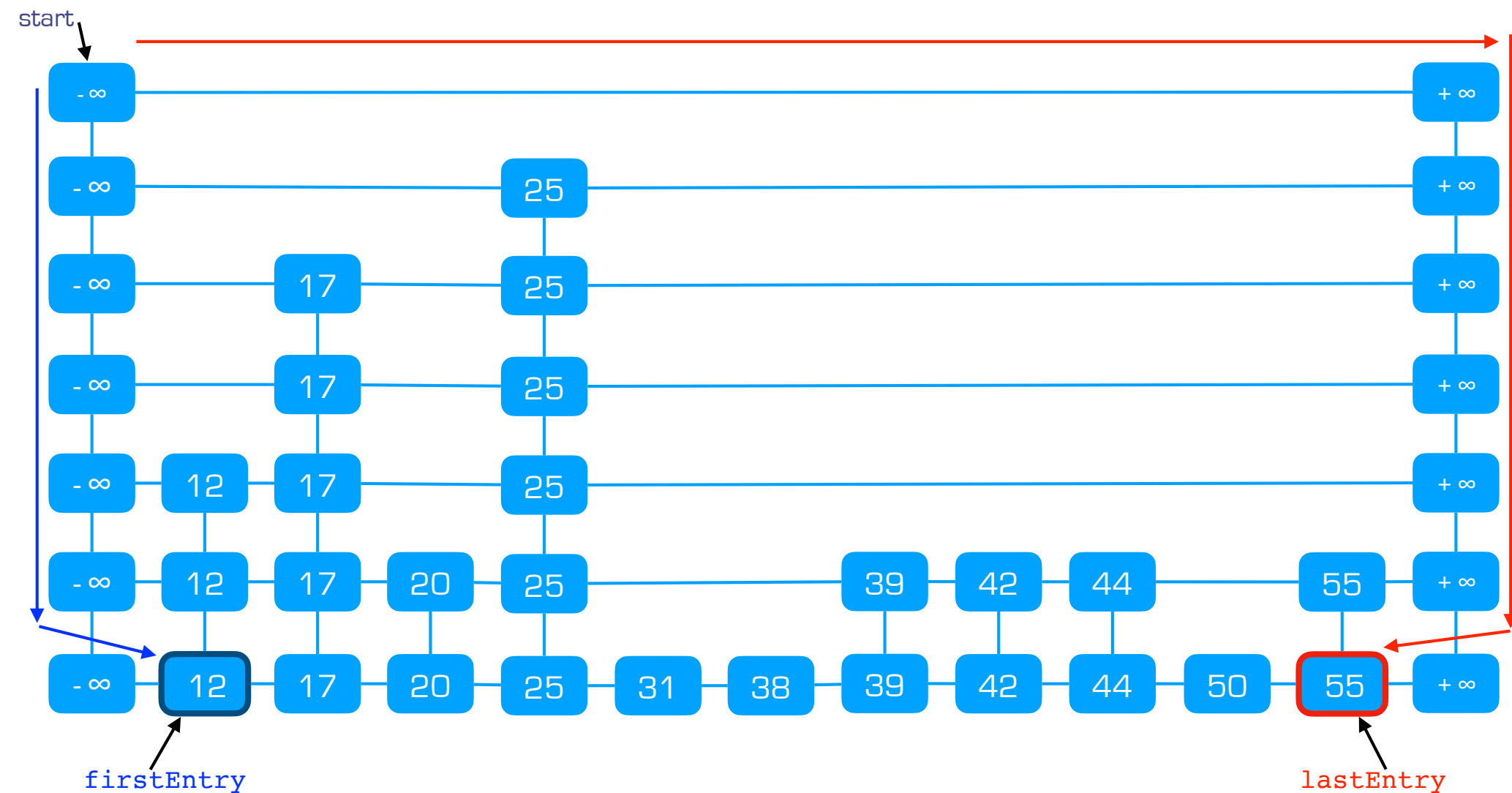
Voir le code de `SkipListMap.java`

Après quelques insertions

```
[java] [<12:12>] (h=2)
[java] [<12:12>, <17:17>] (h=4)
[java] [<12:12>, <17:17>, <55:55>] (h=4)
[java] [<12:12>, <17:17>, <50:50>, <55:55>] (h=4)
[java] [<12:12>, <17:17>, <20:20>, <50:50>, <55:55>] (h=4)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <50:50>, <55:55>] (h=5)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <44:44>, <50:50>, <55:55>] (h=5)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <39:39>, <44:44>, <50:50>, <55:55>] (h=5)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <31:31>, <39:39>, <44:44>, <50:50>, <55:55>] (h=5)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <31:31>, <38:38>, <39:39>, <44:44>, <50:50>, <55:55>] (h=5)
[java] [<12:12>, <17:17>, <20:20>, <25:25>, <31:31>, <38:38>, <39:39>, <42:42>, <44:44>, <50:50>, <55:55>] (h=5)
```

Voir le code de `SkipListMap.java`

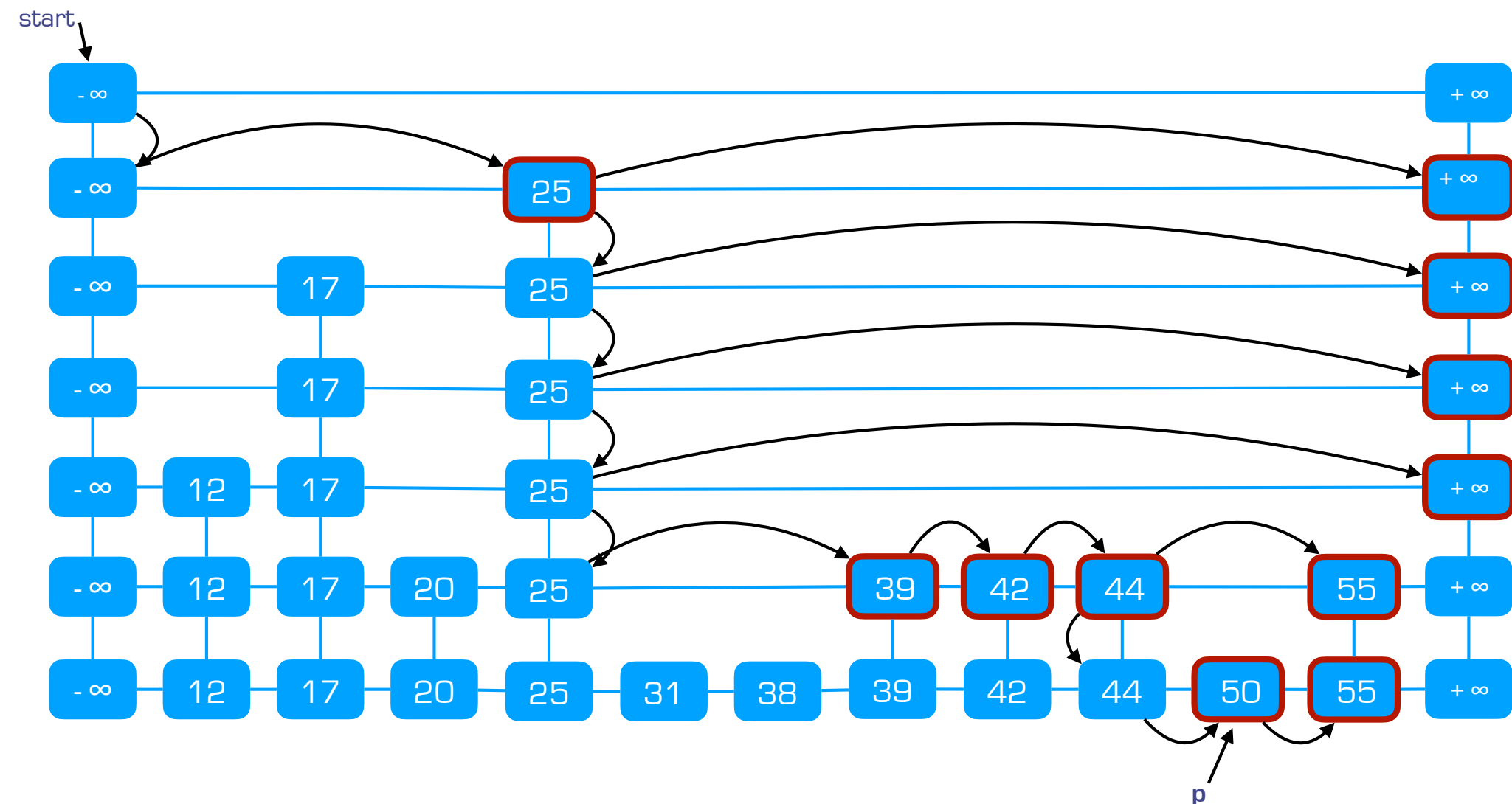
firstEntry et lastEntry



Voir le code de `SkipListMap.java`

Chercher dans une **SkipList** : **skipSearch(50)**

[clé existe assurément au niveau 0; hauteur minimum d'une colonne = 0]

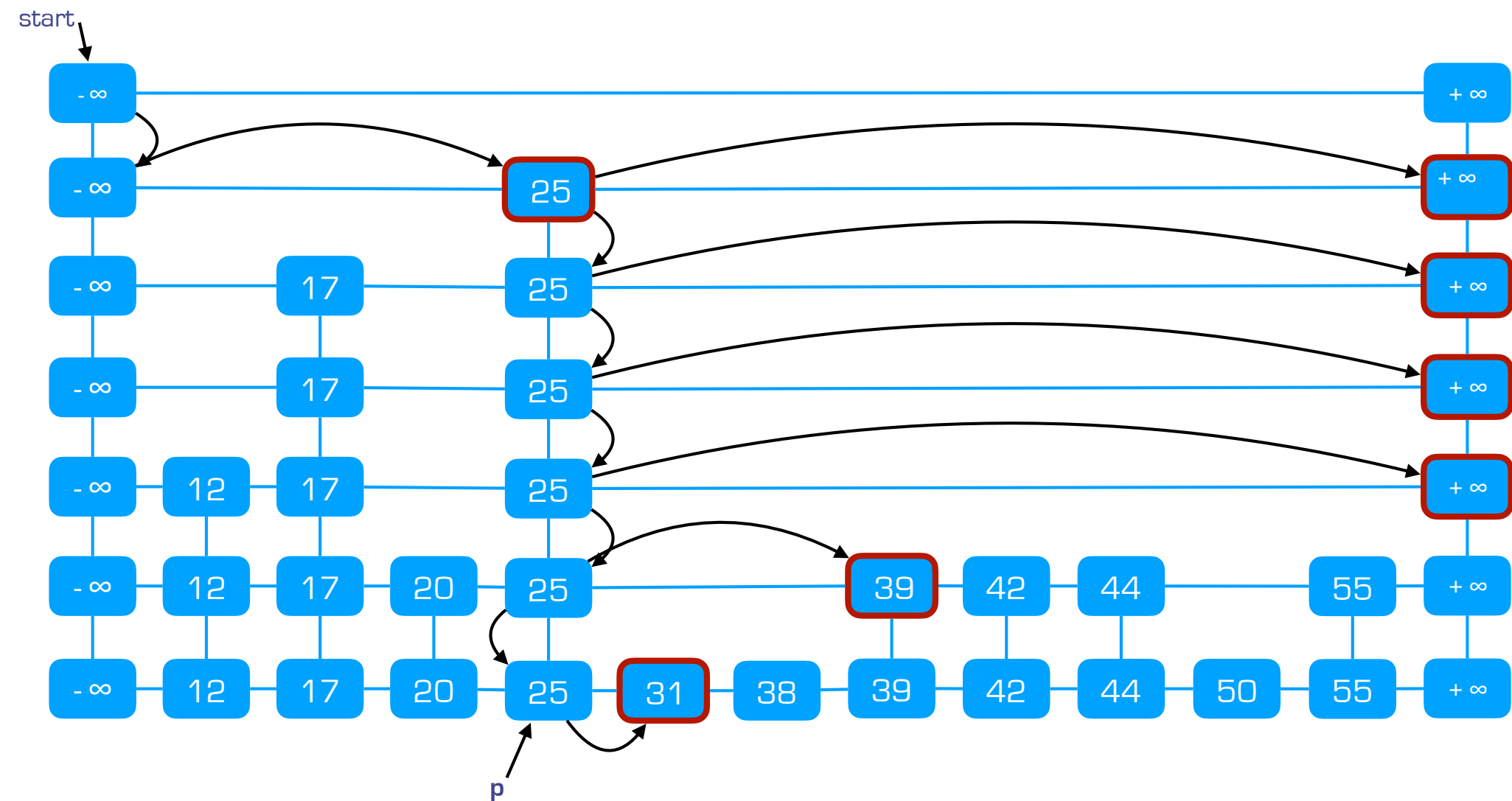
Voir le code de `SkipListMap.java`

11 clés comparées

 clé comparée

Chercher dans une **SkipList** : **skipSearch (25)**

[clé existe assurément au niveau 0; hauteur minimum d'une colonne = 0]

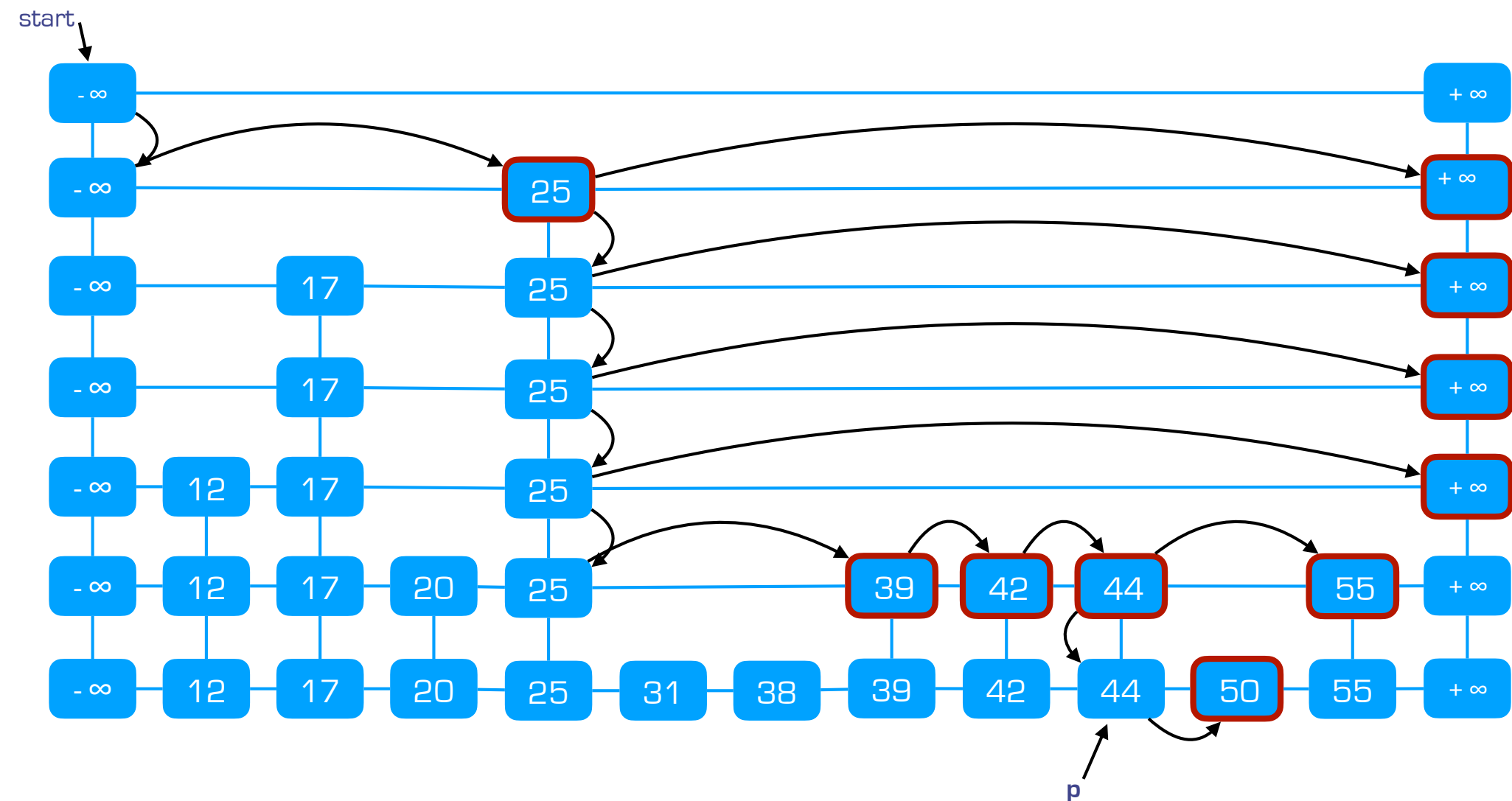
Voir le code de `SkipListMap.java`

7 clés comparées

 clé comparée

Chercher dans une SkipList : `skipSearch( 47 )`

[clé absente, arrête au niveau 0 sur la plus grande clé qui est plus petite que la clé recherchée]

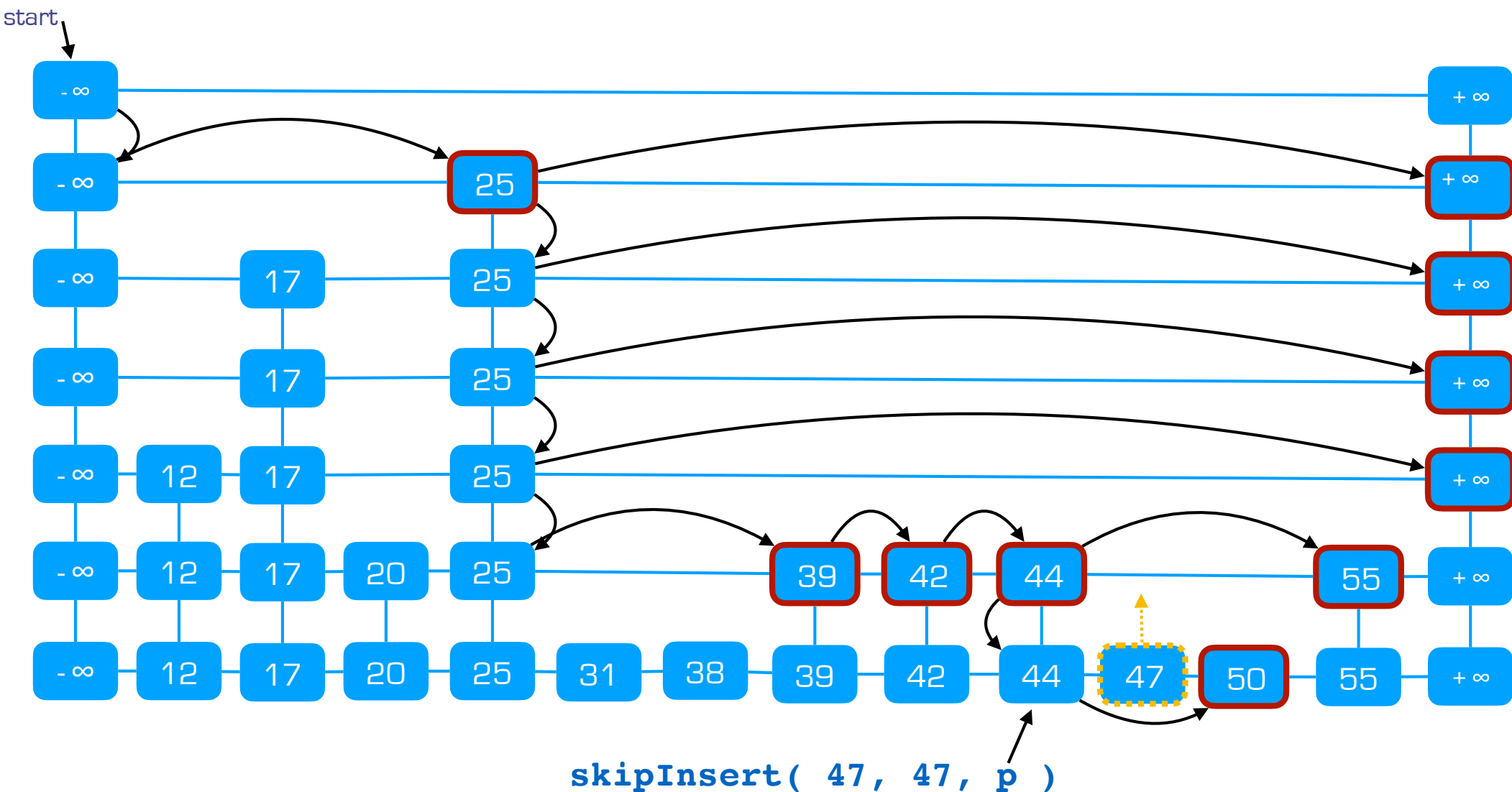


Voir le code de `SkipListMap.java`

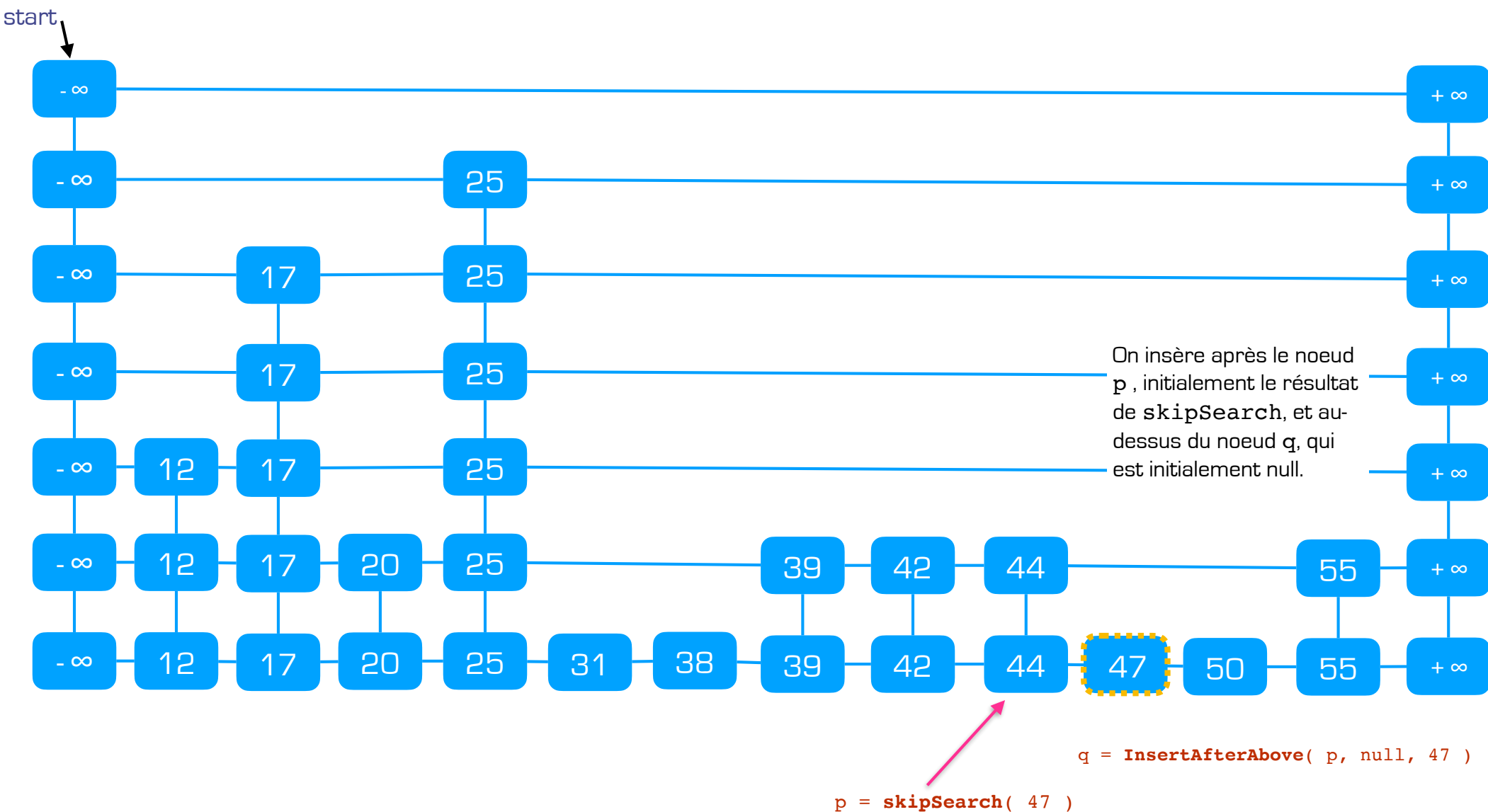
10 clés comparées

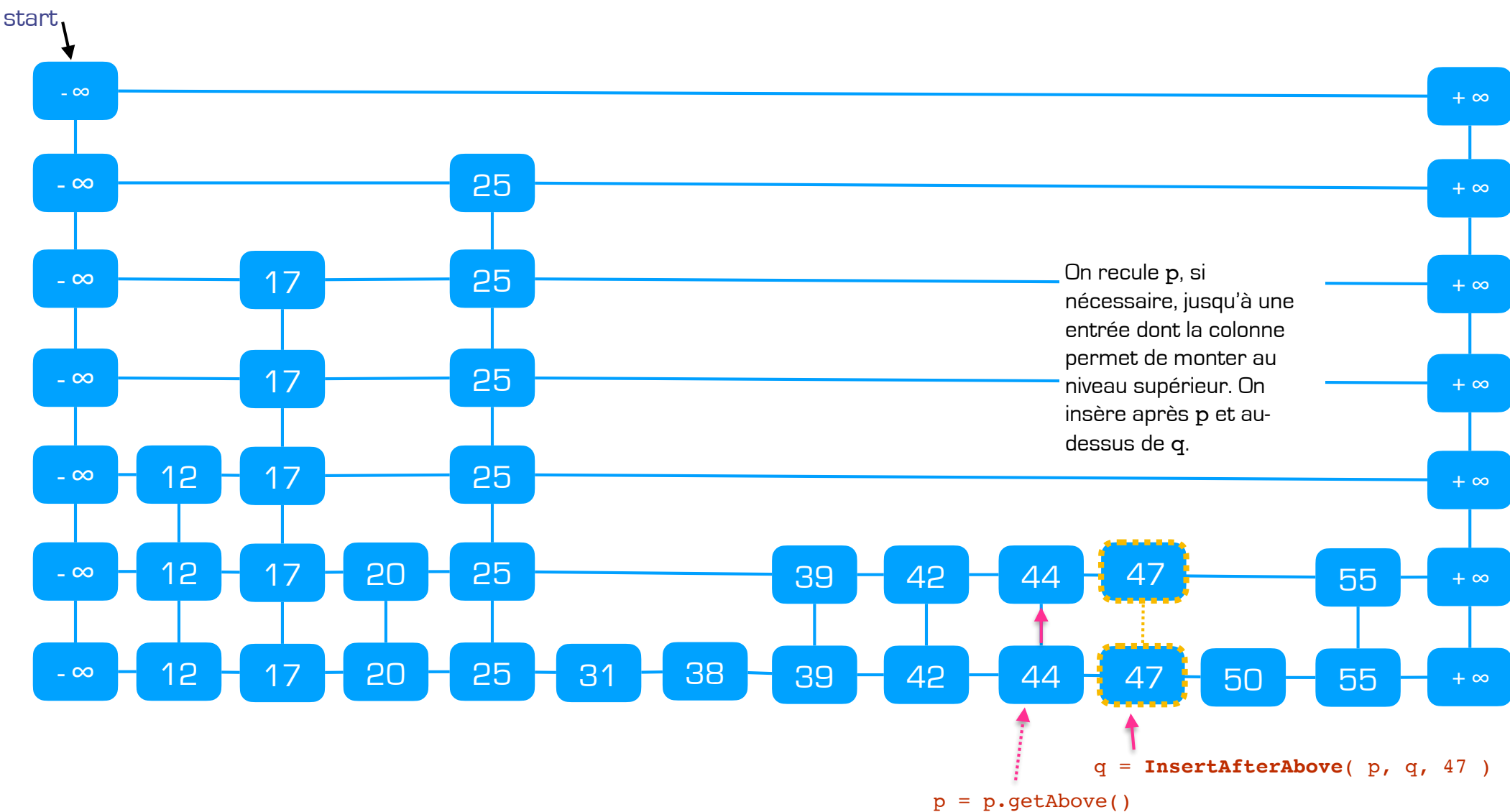
 clé comparée

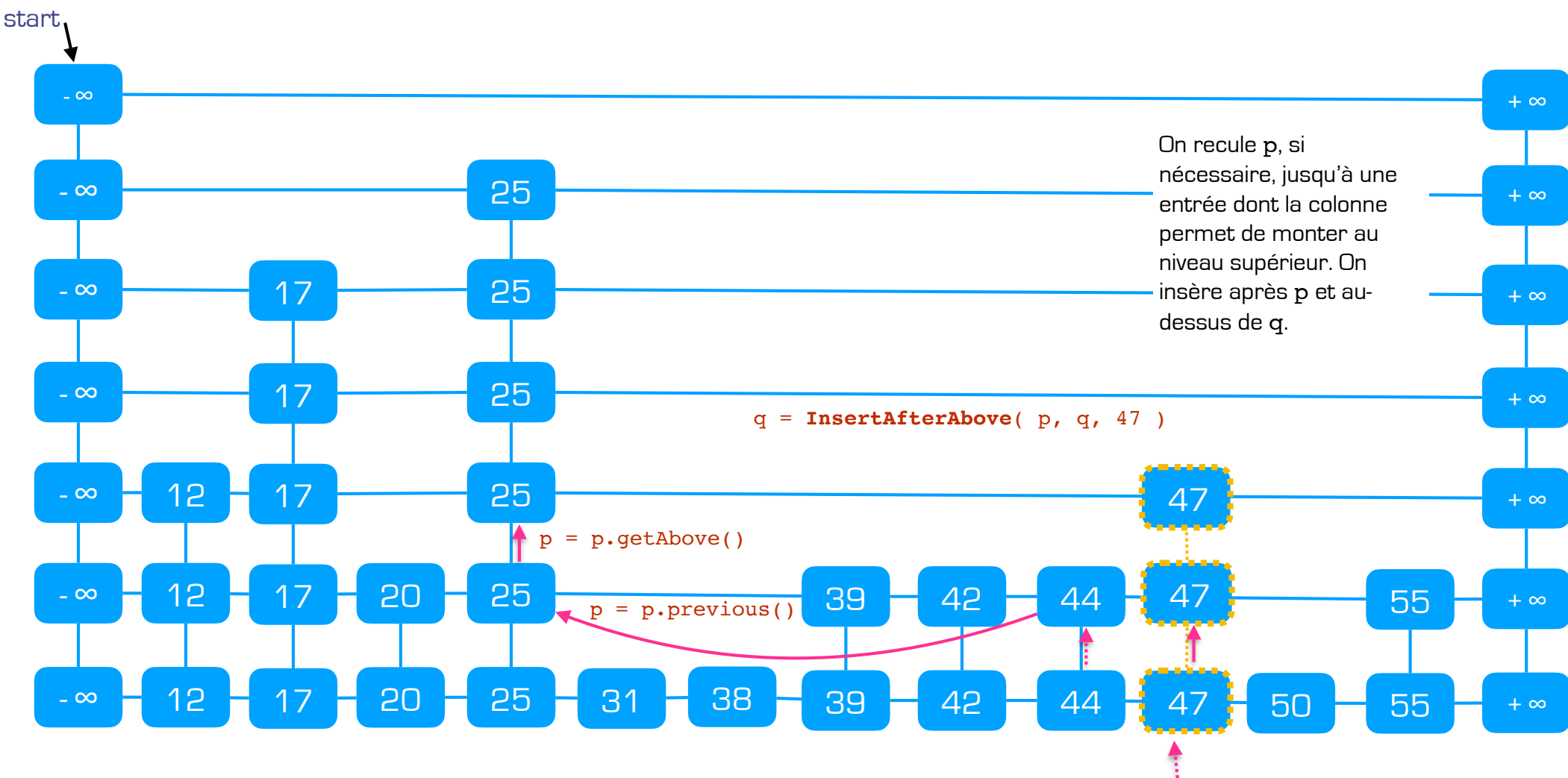
Insérer dans une **SkipList** : `put( 47, 47 )`



Voir le code de `SkipListMap.java`

Insérer dans une **SkipList** : `skipInsert( 47, v, p )`Voir le code de `SkipListMap.java`

Insérer dans une **SkipList** : `skipInsert( 47, 47, p )`Voir le code de `SkipListMap.java`

Insérer dans une **SkipList** : `skipInsert( 47, 47, p )`Voir le code de `SkipListMap.java`

Insérer dans une **SkipList**

On décide la hauteur de la colonne d'une entrée de manière probabiliste, **en jouant à pile ou face**.

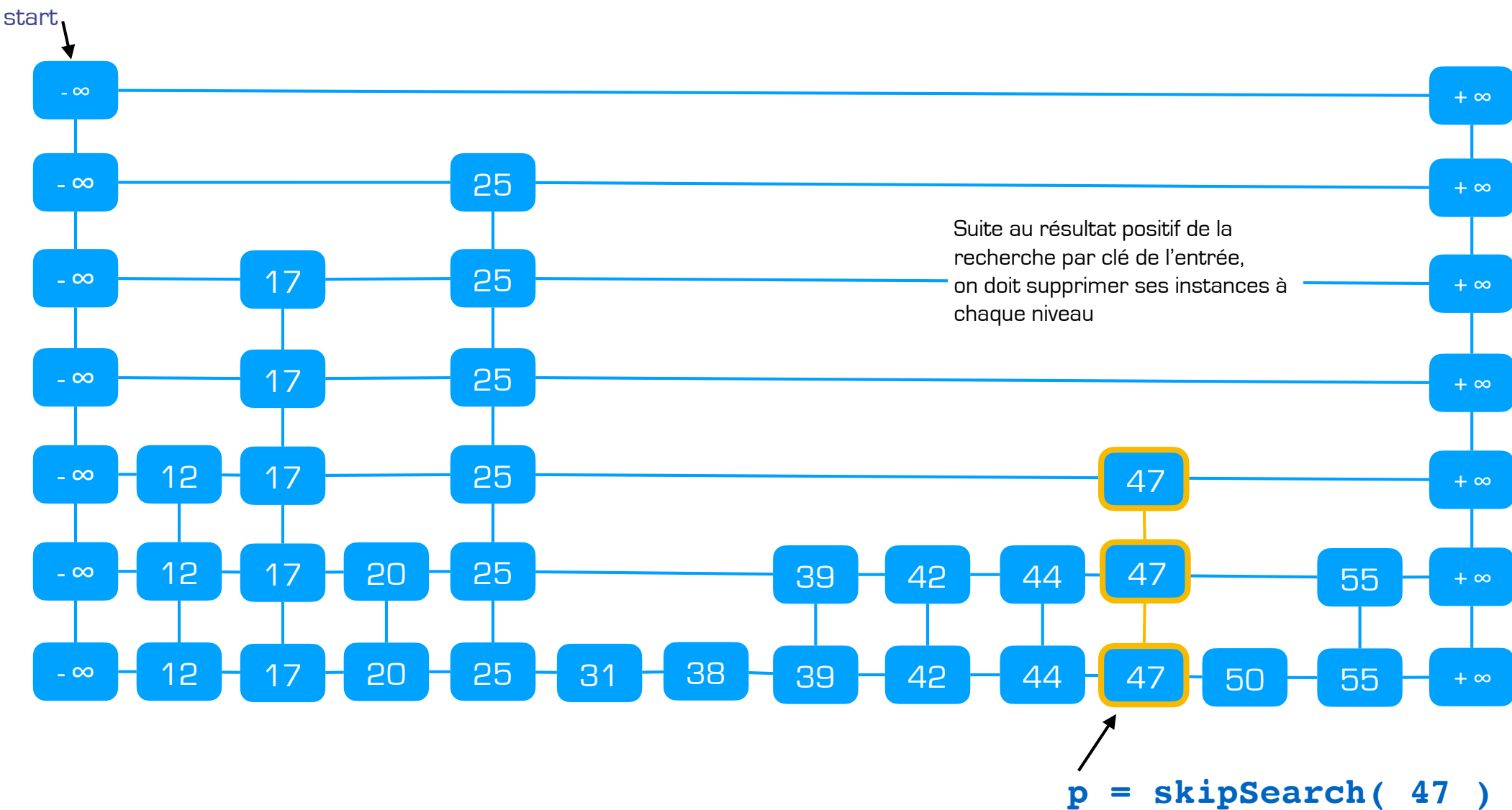
Après l'insertion de la nouvelle entrée (k, v) au niveau 0 , si on a pile, on arrête. Sinon, le tir a donné face et on monte au niveau suivant pour y insérer (k, v) . On lance à nouveau la pièce ; s'il donne face, on monte et insère et ainsi de suite jusqu'à ce que le tir donne pile.

Ainsi, on continue à insérer la nouvelle entrée (k, v) dans les listes jusqu'à ce que nous obtenions un tir pile, ce qui termine l'insertion.

Nous relions toutes les références à la nouvelle entrée (k, v) créée dans ce processus pour créer sa tour.

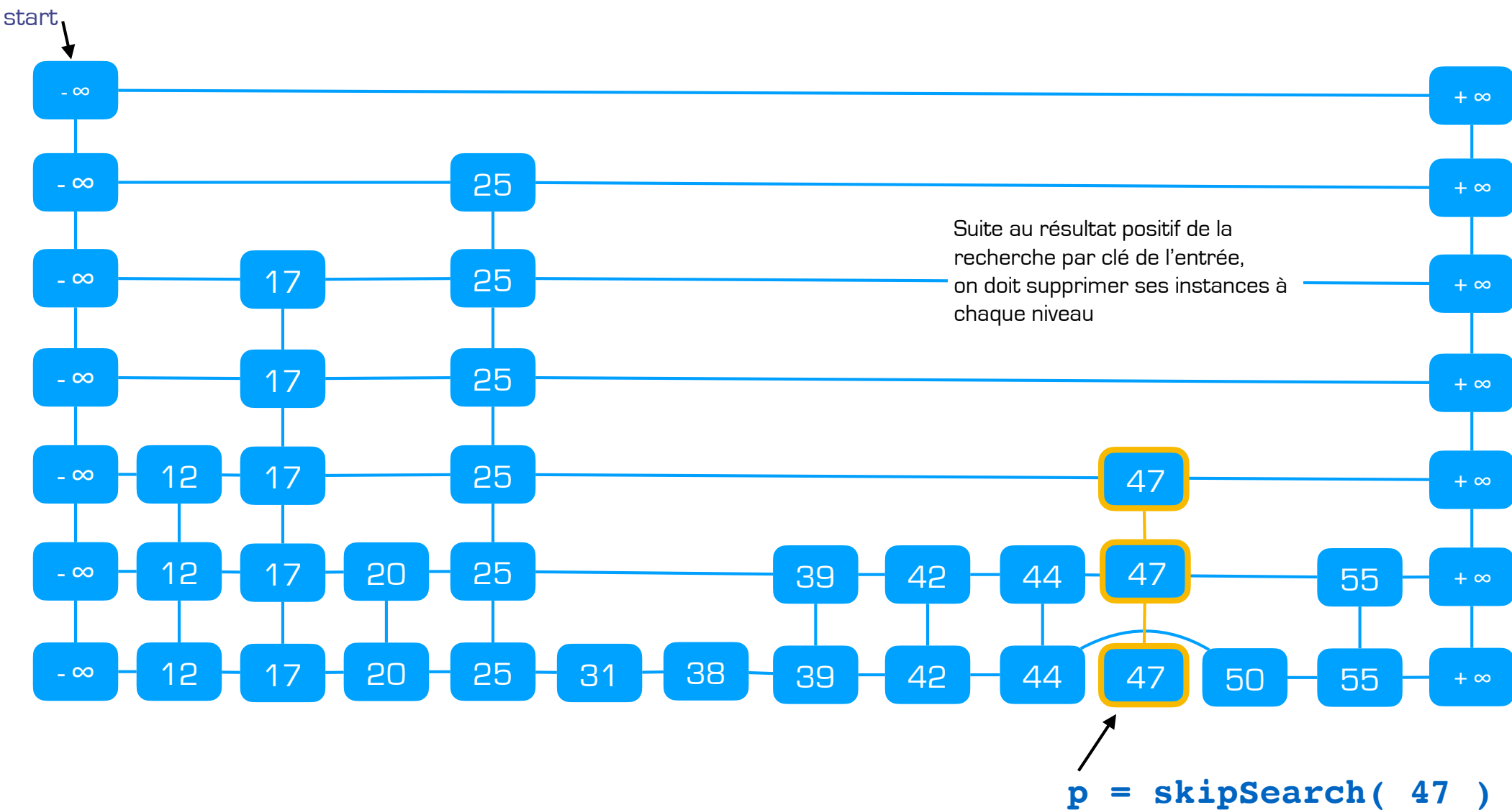
Un bon tirage au sort peut être simulé avec le générateur de nombres pseudo-aléatoires intégré de Java, `java.util.Random`, en appelant `nextBoolean()`, qui renvoie vrai ou faux, chaque fois avec une probabilité de $1/2$.

Voir le code de `SkipListMap.java`

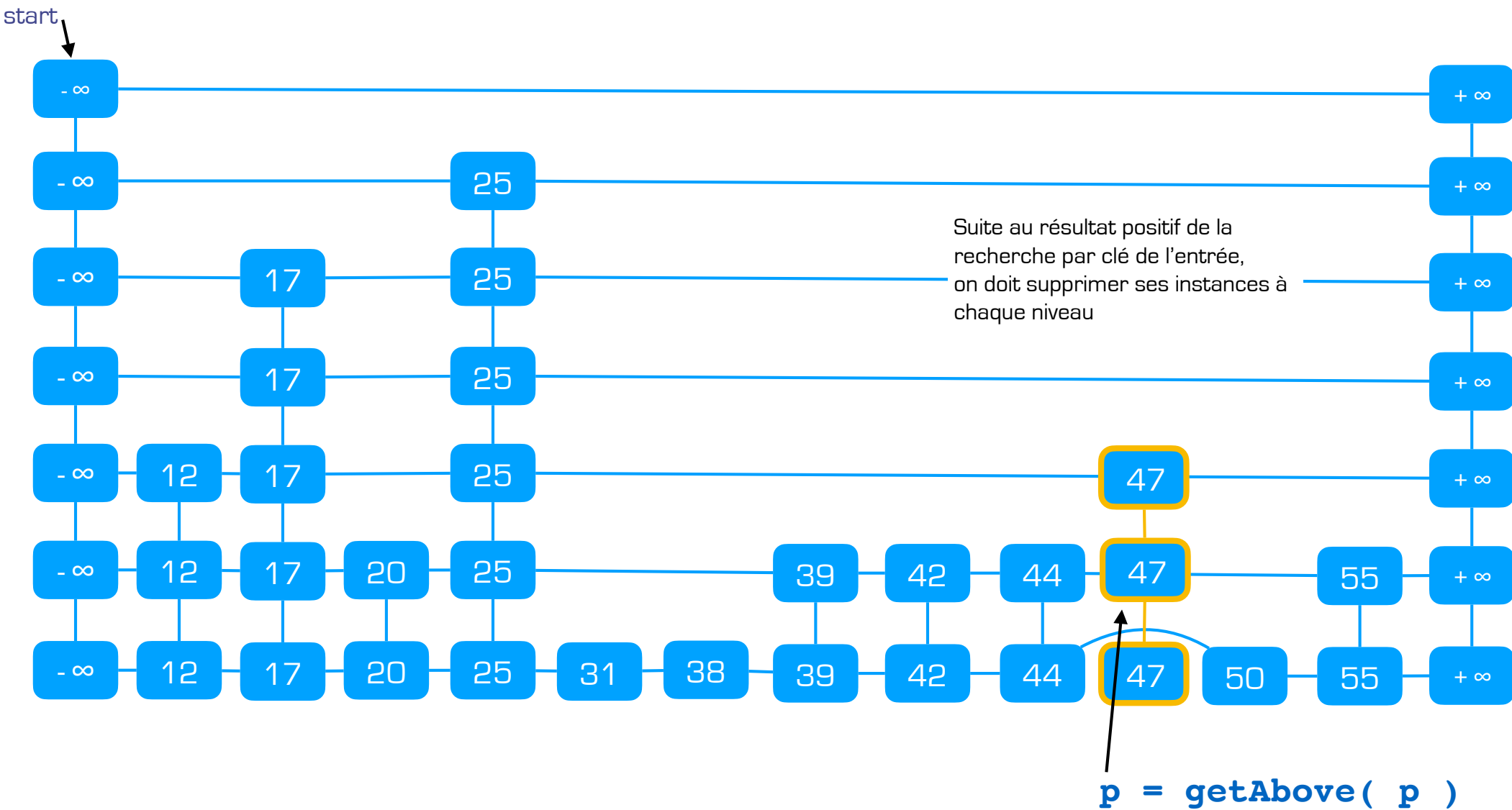
Supprimer dans une **SkipList** : **remove(47)**

Voir le code de `SkipListMap.java`

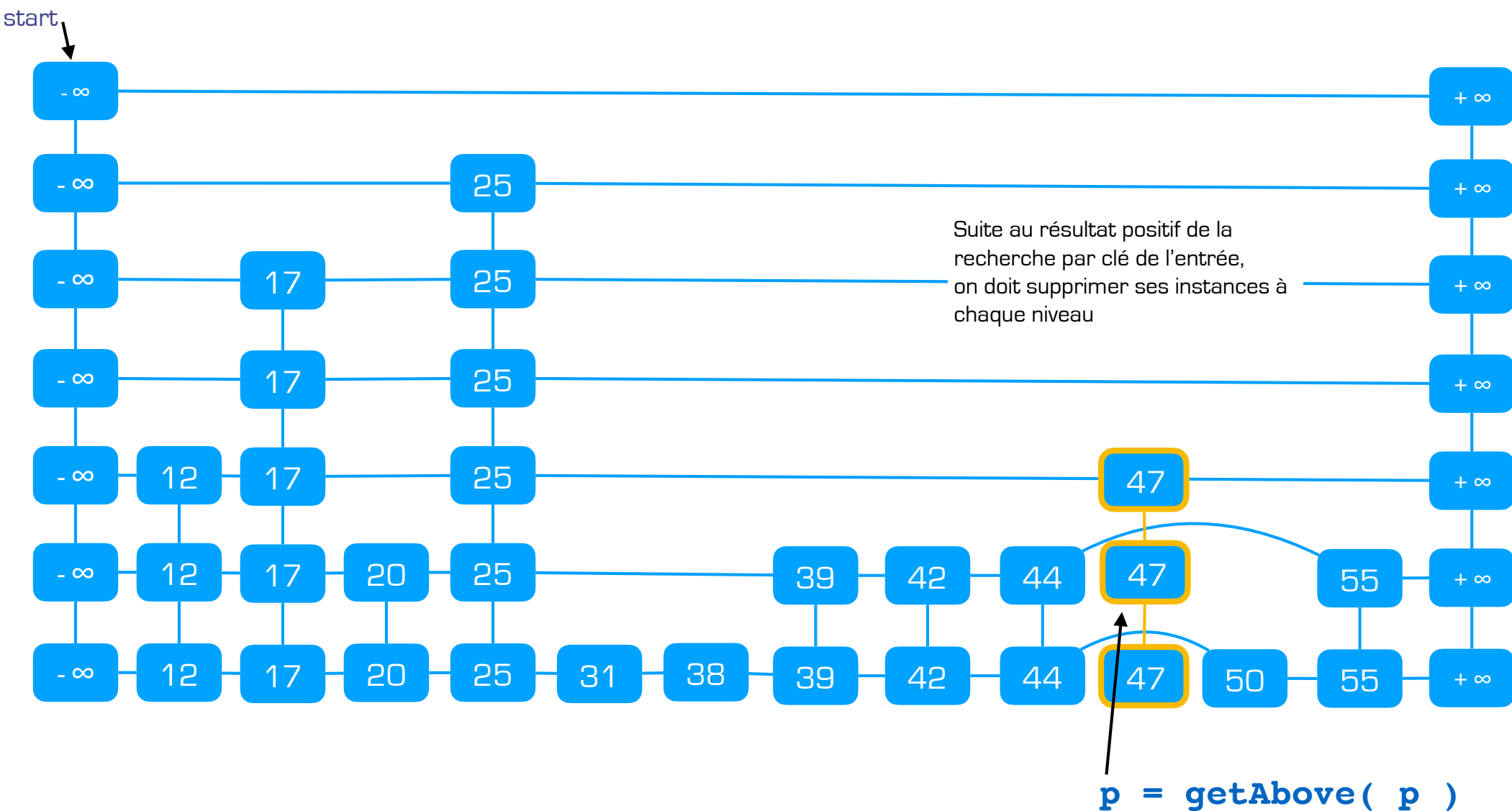
Supprimer dans une **SkipList** : **remove(47)**



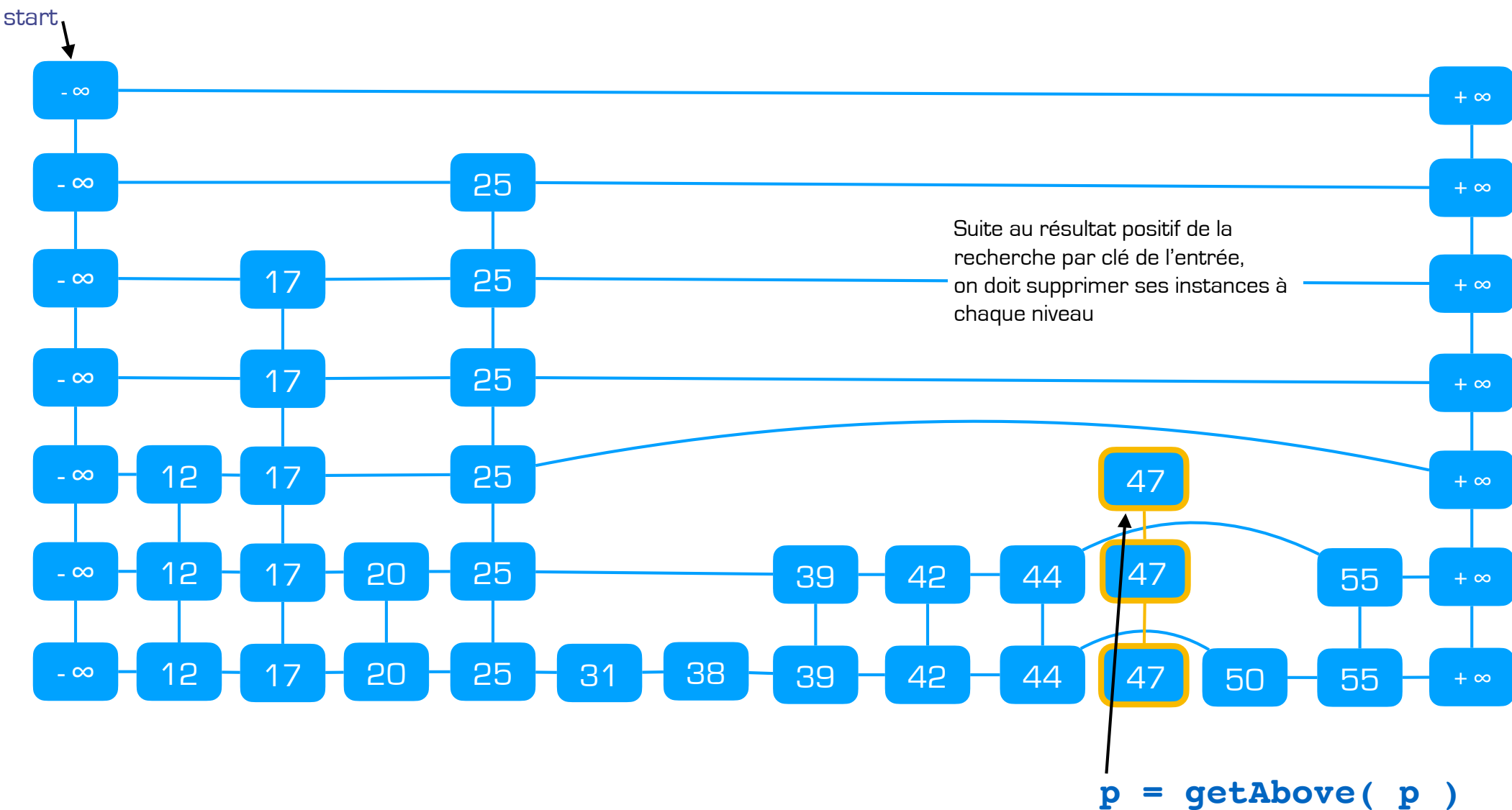
Voir le code de `SkipListMap.java`

Supprimer dans une **SkipList** : **remove(47)**

Voir le code de `SkipListMap.java`

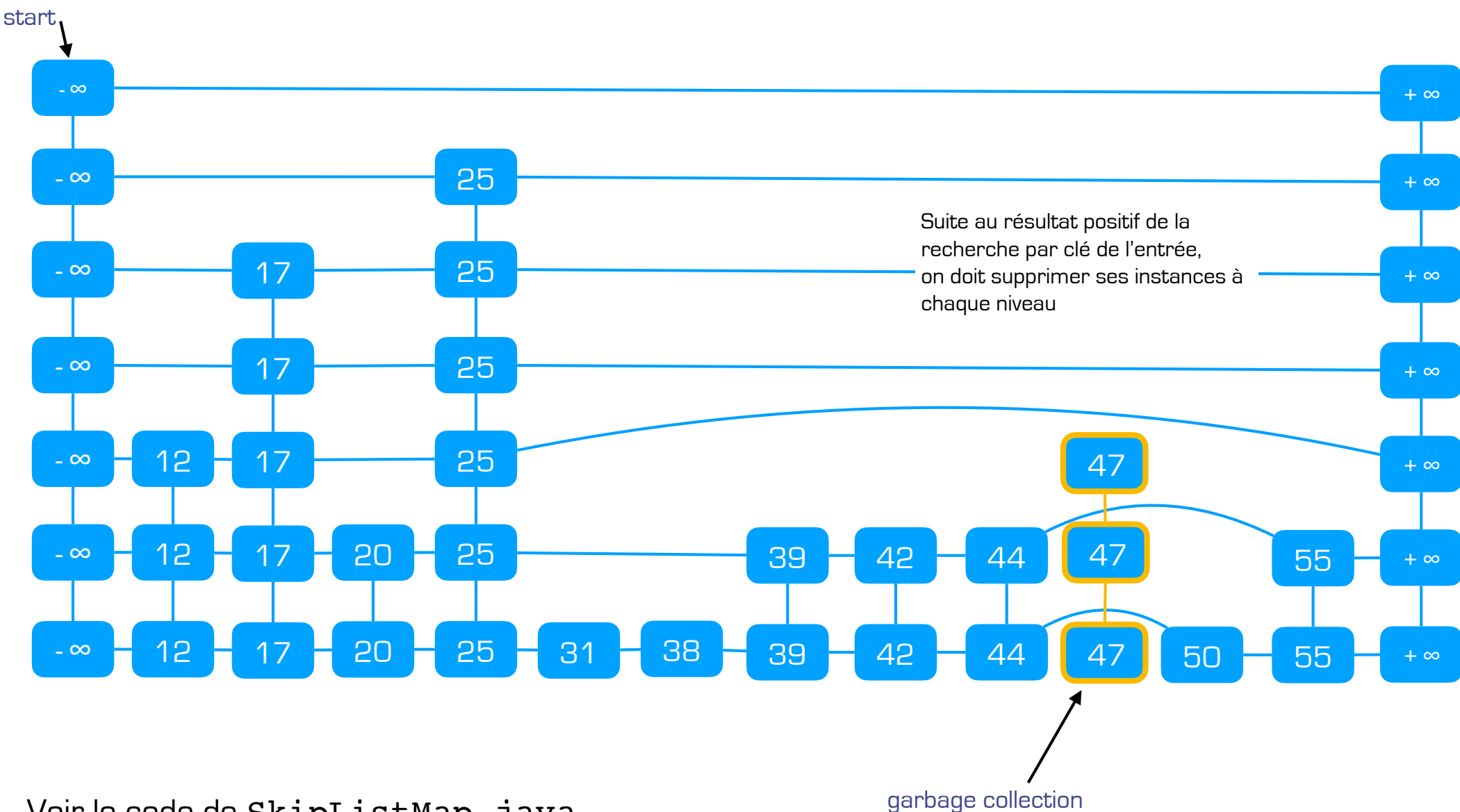
Supprimer dans une **SkipList** : **remove(47)**

Voir le code de `SkipListMap.java`

Supprimer dans une **SkipList** : **remove(47)**

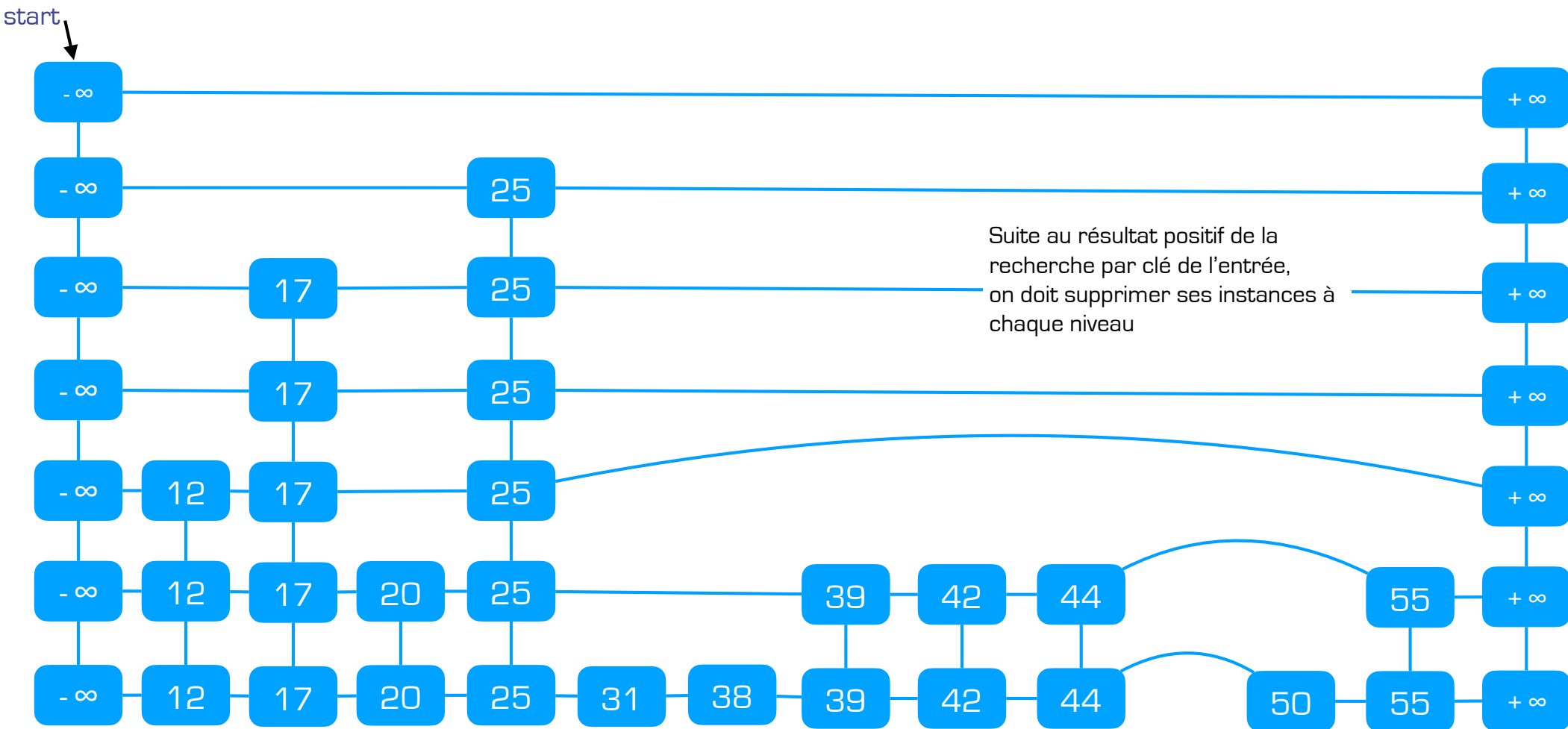
Voir le code de `SkipListMap.java`

Supprimer dans une **SkipList** : **remove(47)**



Voir le code de `SkipListMap.java`

Supprimer dans une **SkipList** : **remove(47)**



Voir le code de `SkipListMap.java`

Performances de **SkipList**

<code>size, isEmpty</code>	$O(1)$
<code>get</code>	$O(\log n)$ exp.
<code>put</code>	$O(\log n)$ exp.
<code>remove</code>	$O(\log n)$ exp.
<code>firstEntry,</code> <code>lastEntry</code>	$O(1)$ accès au premier et dernier élément à partir de <code>start</code> Notre implémentation est en $O(\log n)$
<code>ceilingEntry,</code> <code>floorEntry,</code> <code>lowerEntry,</code> <code>higherEntry</code>	$O(\log n)$ exp.
<code>subMap</code>	$O(s + \log n)$ où s entrées sont retournés et $\log n$ pour la recherche du début de l'intervalle
<code>entrySet,</code> <code>keySet,</code> <code>values</code>	$O(n)$

Maxima-set avec **skipList**

Voir le code de `Trips.java`

```

a [java] {<2500:90>}
b [java] {<2500:90>, <3000:100>}
c [java] {<2500:90>, <3000:100>, <3500:120>}
d [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>}
e [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>}
f [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>}
g [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>, <3700:155>}
h [java] {<2500:90>, <3000:100>, <3500:120>, <3550:130>, <3560:140>, <3600:150>, <3700:155>, <3900:160>}
élimine c, d et e : p [java] {<2500:90>, <3000:100>, <3200:145>, <3600:150>, <3700:155>, <3900:160>}

[java] Proposed trip: <2001:300>
[java] Time to filter 1 dominant trip over 500 millions data points is 17859 milliseconds.
[java] [<2001:300>]

```

17 859 versus 18 106 milliseconds !

➡ Pourquoi si peu de différence entre les implémentations ?

SortedTableMap vs. SkipListMap vs. ProbeHashMap

Data structure 2M entrées	put (sec.)	get (sec.)	remove (sec.)
Sorted list	$O(n)$	$O(\log n)$	$O(n)$
ArrayList	49.7	1.3	42.6
Skip list	$O(\log n)$ exp	$O(\log n)$ exp	$O(\log n)$ exp
SkipListMap	3.1	2.7	2.7
java.util.concurrent. ConcurrentSkipListMap	2.2	1.8	2.2
Hash table	$O(1)$ exp	$O(1)$ exp	$O(1)$ exp
ProbeHashMap	0.4	0.2	0.2